



SMART CONTRACT AUDIT REPORT

for

MAXIBRIDGE



Prepared By: Xiaomi Huang

PeckShield
June 22, 2026

Document Properties

Client	MaxiBridge
Title	Smart Contract Audit Report
Target	MaxiBridge
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Pin Lv, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	June 22, 2026	Xuxian Jiang	Final Release
1.0-rc	June 20, 2026	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1	Introduction	4
1.1	About MaxiBridge	4
1.2	About PeckShield	5
1.3	Methodology	5
1.4	Disclaimer	7
2	Findings	9
2.1	Summary	9
2.2	Key Findings	10
3	Detailed Results	11
3.1	Possibly Premature Fee Withdrawal Before Promotion in BridgeDepository	11
3.2	Improper _signerCount Accounting in setInitialSigner()	12
3.3	Missing Inventory Accounting In Claiming Under Mode 2	13
3.4	Strengthened Flow Activeness Check in provisionInventory()	15
3.5	EVM/OP20 Inconsistency: Epoch Overflow Handling	17
3.6	Suggested EVM Chain ID/evmBridge Validation in addFlow()	18
3.7	Revisited drainInventoryOpNet() Logic in BridgeDepository	20
3.8	Arithmetic Overflow Avoidance in VestingVault	22
3.9	Improved mintTo() Logic in WrappedOP20	23
3.10	Trust Issue of Admin Keys	24
4	Conclusion	27
	References	28

1 | Introduction

Given the opportunity to review the design document and related smart contract source code of the MaxiBridge protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About MaxiBridge

MaxiBridge is a general-purpose token bridge between Ethereum mainnet and OPNet (Bitcoin L1). Governance can onboard almost any token pair through five flow modes (wrapped mint/burn, pooled lock/release, pooled lock/vest) with decimal-aware accounting. Depending on the flow's mode, each side does lock-and-release, mint-and-burn, or pooled custody. Both directions use a voucher model: the user pays gas on the destination chain, not the server. The basic information of the MaxiBridge protocol is as follows:

Table 1.1: Basic Information of The MaxiBridge Protocol

Item	Description
Issuer	MaxiBridge
Type	EVM/OPNet Smart Contract
Platform	Solidity/AssemblyScript
Audit Method	Whitebox
Latest Audit Report	June 22, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- <https://github.com/opnet-collective/MaxiBridge-contracts-audit.git> (754ffec)

And this is the commit ID after all fixes for the issues found in the audit have been checked in:

- <https://github.com/opnet-collective/MaxiBridge-contracts-audit.git> (b75dd84)

1.2 About PeckShield

PeckShield Inc. [11] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/peckshield>), Twitter (<http://twitter.com/peckshield>), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low
		High	Medium	Low
		Likelihood		

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	OP20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [9], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the MaxiBridge implementation. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	3	■ ■ ■
Low	7	■ ■ ■ ■ ■ ■ ■
Informational	0	
Total	10	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in [Section 3](#).

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 3 medium-severity vulnerabilities and 7 low-severity vulnerabilities.

Table 2.1: Key MaxiBridge Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Medium	Possibly Premature Fee Withdrawal Before Promotion in BridgeDepository	Business Logic	Resolved
PVE-002	Medium	Improper <code>_signerCount</code> Accounting in <code>setInitialSigner()</code>	Coding Practices	Resolved
PVE-003	Low	Missing Inventory Accounting In Claiming Under Mode 2	Business Logic	Resolved
PVE-004	Low	Strengthened Flow Activeness Check in <code>provisionInventory()</code>	Business Logic	Resolved
PVE-005	Low	EVM/OP20 Inconsistency: Epoch Overflow Handling	Business Logic	Resolved
PVE-006	Low	Suggested EVM Chain ID/ <code>evmBridge</code> Validation in <code>addFlow()</code>	Coding Practices	Resolved
PVE-007	Low	Revisited <code>drainInventoryOpNet()</code> Logic in BridgeDepository	Business Logic	Resolved
PVE-008	Low	Arithmetic Overflow Avoidance in VestingVault	Coding Practices	Resolved
PVE-009	Low	Improved <code>mintTo()</code> Logic in WrapperOP20	Business Logic	Resolved
PVE-010	Medium	Trust Issue of Admin Keys	Security Features	Resolved

Besides recommending specific countermeasures to mitigate these issues, we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for details.

3 | Detailed Results

3.1 Possibly Premature Fee Withdrawal Before Promotion in BridgeDepository

- ID: PVE-001
- Severity: Medium
- Likelihood: Low
- Impact: Medium
- Target: BridgeDepository
- Category: Business Logic [8]
- CWE subcategory: CWE-841 [5]

Description

In the MaxiBridge protocol, there is a core BridgeDepository contract on the OPNet side of the bridge. Each bridge request may be charged with a fee based on the associated flow and the accrued fee may be later withdrawn by the protocol governor or guardian. In the process of reviewing the bridge fee withdrawal logic, we notice the bridge fee may be collected before fully promoted.

To elaborate, we show below the code snippet from the related `withdrawFees()` function. This function considers all fee accrued in `this._flowAccruedFees.get(flowId)` (line 2347) withdrawable, i.e., fully promoted. However, when a bridge request is locked, the fee may not be promoted until settled. As a result, a pre-promoted bridge fee may be withdrawn, but it can be later requested for refund. To fully resolve this issue, we can mirror the EVM-side logic by introducing the (missing) `SETTLED` state for each bridge lock request. And the associated bridge fee may be promoted only after fully settled. The following `withdrawFees()` function can only withdraw settled bridge fees.

```
2325 public withdrawFees(calldata: Calldata): BytesWriter {
2326     // Governor OR guardian (mirrors EVM 'BridgeEscrow.withdrawFees'
2327     // onlyOwnerOrGuardian).
2328     this.onlyGovernorOrGuardian();
2329     const flowId: u256 = calldata.readU256();
2330     const token: Address = calldata.readAddress();
2331     const amount: u256 = calldata.readU256();
2332
2333     if (token.isZero()) throw new Revert('BridgeDepository: zero token');
```

```

2334     if (amount.isZero()) throw new Revert('BridgeDepository: zero amount');
2335     if (this._flowExists.get(flowId).isZero()) {
2336         throw new Revert('BridgeDepository: flow not found');
2337     }
2338     // Bind the passed token to the flow's registered canonical token -
2339     // a withdrawal can't be made against a different asset.
2340     if (!u256.eq(this._flowOpnetToken.get(flowId), _opnetAddrToU256(token))) {
2341         throw new Revert('BridgeDepository: token not flow canonical');
2342     }

2344     // Verify -> effect -> interaction (CEI). The accumulator is the sole
2345     // bound; decrement it BEFORE the external transfer. An over-withdraw
2346     // reverts before any token moves.
2347     const accrued: u256 = this._flowAccruedFees.get(flowId);
2348     if (u256.gt(amount, accrued)) {
2349         throw new Revert('BridgeDepository: insufficient accrued fees');
2350     }
2351     this._flowAccruedFees.set(flowId, SafeMath.sub(accrued, amount));
2352     ...
2353 }

```

Listing 3.1: BridgeDepository::withdrawFees()

Recommendation Revise the above-mentioned routine to ensure only promoted bridge fees may be withdrawn.

Status The issue has been resolved by the following commit: 122b0b6.

3.2 Improper `_signerCount` Accounting in `setInitialSigner()`

- ID: PVE-002
- Severity: Medium
- Likelihood: Low
- Impact: Medium
- Target: BridgeDepository
- Category: Coding Practices [7]
- CWE subcategory: CWE-1099 [1]

Description

As mentioned earlier, in `MaxiBridge`, both bridge directions use a voucher model that requires multiple signers to sign on an intended operation. And there is a storage state of `signerCount` to record the number of authorized signers. While reviewing a specific routine to set the signer pubkey for the current epoch on the `OPNET`-side, this state may not be properly maintained.

To elaborate, we show below the implementation of the related `setInitialSigner()` routine, which has a rather straightforward logic to configure the signer pubkey for the current epoch. However, it comes to our attention that it blindly advances the `signerCount` state by 1 (line 900) without

validating whether the added one is a new one. If the added signer is an existing one, there is no need to update `signerCount` at all.

```

885     public setInitialSigner(calldata: Calldata): BytesWriter {
886         this.onlyGovernor();
887         const pubKey: Uint8Array = calldata.readBytesWithLength();
888         if (pubKey.length === 0) {
889             throw new Revert('BridgeDepository: empty signer pubkey');
890         }
891         const epoch: u256 = this._signerEpoch.value;
892         if (!this._bridgeSignerHashes.get(epoch).isZero()) {
893             throw new Revert('BridgeDepository: signer already set - use rotateSigner');
894         }
895         const signerHash: u256 = u256.fromUint8ArrayBE(sha256(pubKey));
896         this._bridgeSignerHashes.set(epoch, signerHash);
897
898         // v2 - seed the M-of-N set so claim() finds the signer.
899         this._signerKeyHashSet.set(signerHash, u256.One);
900         this._signerCount.value = SafeMath.add(this._signerCount.value, u256.One);
901         if (this._requiredSignatures.value.isZero()) {
902             this._requiredSignatures.value = u256.One;
903         }
904
905         this.emitEvent(new SignerRotated(epoch.toU32(), epoch.toU32(), signerHash));
906         return new BytesWriter(0);
907     }

```

Listing 3.2: BridgeDepository:: setInitialSigner ()

Recommendation Revisit the above logic to ensure the `signerCount` state is only updated when the added signer for the current epoch is a new one.

Status The issue has been resolved by the following commit: 75f6d13.

3.3 Missing Inventory Accounting In Claiming Under Mode 2

- ID: PVE-003
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: BridgeDepository
- Category: Business Logic [8]
- CWE subcategory: CWE-841 [5]

Description

The MaxiBridge protocol organizes the bridge requests into various flows and each flow is considered as the first-class object representing a single bridging route. This object is internally represented with the `FlowRecord` structure. Inside this data struct, there is a core field `cap` to denote the intended

ceiling. We notice this `cap` field denotes total inventory ceiling for the flow mode of 0/3/4 or mint ceiling for the flow mode of 2. Our analysis shows that it indeed represents the total inventory ceiling for the flow mode of 0/3/4, but it does not properly enforce the mint ceiling for the mode of 2.

```

233     struct FlowRecord {
234         // Set-once at addFlow:
235         uint8    mode;                // 0=WRAPPED, 1=INVERSE_WRAPPED, 2=NATIVE_BURN_MINT,
                3=POOLED_LOCK_RELEASE, 4=POOLED_LOCK_VEST
236         uint8    status;              // see FLOW_STATUS_* above
237         uint64    evmChainId;
238         address   evmBridge;           // BridgeEscrow proxy on the EVM source chain
239         address   evmToken;
240         uint8     evmDecimals;
241         uint8     opnetDecimals;
242         bytes32   opnetBridge;         // 32-byte canonical OPNet address
243         bytes32   opnetToken;
244         // Mutable via timelocked governance:
245         uint16    feeBps;               // 0..MAX_FEE_BPS
246         uint128   minFee;              // dst-side base units
247         uint128   minAmount;           // EVM-source base units
248         uint128   cap;                 // total inventory ceiling (mode 0/3/4) or mint
                ceiling (mode 2)
249         uint128   dailyLimit;           // per-flow rolling window
250         // Hot fields written by claim/release in PR
251         uint128   mintedToday;
252         uint64    lastWindowStart;
253         uint128   inventory;           // mode 0/3/4: locked tokens; mode 2: synthetic supply
254         // PR - per-flow permissionless relayer tip cap.
255         // 0 = tipping disabled (default); cap is per-flow,
256         // <= MAX_TIP_BPS = 200 (2%); governor-set.
257         uint16    tipCapBps;
258         // Mode 4 (POOLED_LOCK_VEST) destination vault. Required iff
259         // 'mode == POOLED_LOCK_VEST'; address(0) for every other mode. On
260         // claim, the bridge approves and calls 'depositFor' on this vault
261         // instead of 'safeTransfer'ing directly to the recipient. Governor-
262         // repointable via 'setFlowVestingVault' (restricted to mode 4).
263         address   vestingVault;
264         // Per-flow accumulated source-side bridge fees (EVM-lock direction).
265         // Incremented on each 'lock' by exactly the computed fee portion;
266         // withdrawn (treasury-only) via 'withdrawFees'. This is the entire
267         // safety bound for routine fee collection - 'withdrawFees' can NEVER
268         // exceed this accumulator, so it can never touch user-locked
269         // principal or another flow's reserves. Struct-field append (the
270         // struct lives in a mapping) - does NOT consume a top-level storage
271         // slot, so the 'uint256[40] __gap' is unaffected.
272         uint128   accruedFees;
273     }

```

Listing 3.3: Per-flow record `FlowRecord` in `BridgeEscrow`

Specifically, if we examine the specific function of `refundBurn()`, when a `WrappedERC20` burn's OPNet destination voucher is permanently cancelled, the burned tokens are gone and the burner may

never get refunded. This `refundBurn()` function is designed to re-mint the burned `WrappedERC20` token amount back to the original burner. However, in current implementation, this function does not properly validated against the above `cap` field.

Recommendation Revisit the `cap` field in `FlowRecord` struct to ensure it properly honors the mint ceiling. In the meantime, we notice the `WrappedERC20` token has the built-in `MAX_SUPPLY` state to gate as the mint ceiling.

Status The issue has been resolved by the following commits: `c8d9d8f` and `823e704`.

3.4 Strengthened Flow Activeness Check in `provisionInventory()`

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `BridgeEscrow`, `BridgeDepository`
- Category: Business Logic [8]
- CWE subcategory: CWE-841 [5]

Description

As mentioned in Section 3.3, `MaxiBridge` organizes the bridge requests into various flows for accounting. Specifically, a flow is the smallest atom of bridge routing — defined by `(mode, evmChainId, evmBridge, evmToken, opnetBridge, opnetToken)`. Vouchers are validated against the flow's status before any inventory mutation. In total, there are five flow status: `disabled`, `active`, `paused`, `draining`, and `retired`. In the analysis of the logic to add inventory to a specific flow on both `EVM` and `OPNET` sides, we notice current logic does not validate the flow status, even though the inventory may only be provisioned in the `active` and `draining` status only.

In the following, we use the `EVM` side as an example and show the implementation of the related function, i.e., `provisionInventory()`. It has a straightforward logic in validating the given flow mode and token amount and then transferring into the contract from the caller before updating the flow's inventory. However, it does not validate the flow status.

```

1750     function provisionInventory(bytes32 flowId, uint256 amount) external onlyOwner
1751         nonReentrant {
1752             if (amount == 0) revert AmountZero();
1753             FlowRecord storage flow = _requireFlow(flowId);
1754             if (
1755                 flow.mode != uint8(TokenMode.WRAPPED)
1756                 && flow.mode != uint8(TokenMode.POOLED_LOCK_RELEASE)
1757                 && flow.mode != uint8(TokenMode.POOLED_LOCK_VEST)
1758             ) {
1759                 revert WrongMode();
1760             }

```

```

1760 // Mode 4 - prevent provisioning a flow whose VestingVault hasn't
1761 // been wired yet. Symmetric to the lock-side guard; closes the
1762 // last path that could otherwise sit inventory on a half-set-up
1763 // flow.
1764 if (flow.mode == uint8(TokenMode.POOLED_LOCK_VEST) && flow.vestingVault ==
    address(0)) {
1765     revert VestingVaultNotSet();
1766 }

1768 address token = flow.evmToken;
1769 IERC20 erc20 = IERC20(token);
1770 uint256 balBefore = erc20.balanceOf(address(this));
1771 erc20.safeTransferFrom(msg.sender, address(this), amount);
1772 uint256 received;
1773 unchecked {
1774     received = erc20.balanceOf(address(this)) - balBefore;
1775 }
1776 if (received == 0) revert NothingReceived();
1777 if (received > type(uint128).max) revert FlowCapExceeded();

1779 // Atomic: tokens in AND accounting up, in the same call.
1780 uint256 newInventory = uint256(flow.inventory) + received;
1781 if (newInventory > uint256(flow.cap)) revert FlowCapExceeded();
1782 // cap is uint128 -> newInventory <= cap <= uint128.max
1783 // forge-lint: disable-next-line(unsafe-typecast)
1784 flow.inventory = uint128(newInventory);

1786 emit InventoryProvisioned(flowId, token, msg.sender, received);
1787 }

```

Listing 3.4: BridgeEscrow::provisionInventory()

Recommendation Revise the above-mentioned routines to ensure the inventory may be provisioned only in the active and draining status. Note it affects the logic on both EVM and OPNET sides.

Status The issue has been resolved by the following commit: 75f6d13.

3.5 EVM/OP20 Inconsistency: Epoch Overflow Handling

- ID: PVE-005
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: BridgeEscrow, BridgeDepository
- Category: Coding Practices [7]
- CWE subcategory: CWE-1099 [1]

Description

Both sides of `MaxiBridge` are designed to function symmetrically, including the role assignment, exported functionality, emitted events etc. In the meantime, the attached blockchain may exhibit unique characteristics (e.g., epoch schedules and block times). In the process of reviewing the symmetric logic of both sides, we notice certain inconsistency that may be removed.

For example, on the EVM side, the related `BridgeEscrow` contract has a function – `removeSigner()` – that is designed to remove an existing signer. However, it silently wraps current epoch if there is an overflow of `newEpoch = oldEpoch + 1` (line 2435). While this overflow is practically unlikely, this behavior is inconsistent with the counterpart on the `OPNET` side, which simply reverts if this (unlikely) overflow occurs. Note this inconsistency also occurs in the following functions, i.e., `setRequiredSignatures()` and `setThreshold()`.

```

1423     function removeSigner(address oldSigner) external onlyOwnerOrGuardian {
1424         if (!isSigner[oldSigner]) revert NotASigner();
1425         unchecked {
1426             uint256 newCount = signerCount - 1;
1427             if (newCount < signerThreshold) revert InvalidThreshold();
1428             signerCount = newCount;
1429         }
1430         isSigner[oldSigner] = false;

1432         uint32 oldEpoch = currentEpoch;
1433         uint32 newEpoch;
1434         unchecked {
1435             newEpoch = oldEpoch + 1;
1436         }
1437         currentEpoch = newEpoch;
1438         emit SignerRemoved(oldSigner, signerCount);
1439         emit SignerEpochRotated(oldEpoch, newEpoch, oldSigner);
1440     }
1441     function removeSigner(address oldSigner) external onlyOwnerOrGuardian {
1442         if (!isSigner[oldSigner]) revert NotASigner();
1443         unchecked {
1444             uint256 newCount = signerCount - 1;
1445             if (newCount < signerThreshold) revert InvalidThreshold();
1446             signerCount = newCount;
1447         }

```

```

1448     isSigner[oldSigner] = false;

1450     uint32 oldEpoch = currentEpoch;
1451     // PVE005 - fail-closed at u32 epoch exhaustion (parity with OPNet
1452     // 'removeSignerFromSet'). Unreachable in practice (2^32 rotations) but
1453     // the silent wrap on overflow would reuse an old epoch.
1454     if (oldEpoch == type(uint32).max) revert SignerEpochExhausted();
1455     uint32 newEpoch;
1456     unchecked {
1457         newEpoch = oldEpoch + 1;
1458     }
1459     currentEpoch = newEpoch;
1460     emit SignerRemoved(oldSigner, signerCount);
1461     emit SignerEpochRotated(oldEpoch, newEpoch, oldSigner);
1462 }

```

Listing 3.5: Staking::removeSigner()

Moreover, both sides export the function that allows an authorized entity to drain the inventory from respective flows. However, the OPNET side allows to specify a recipient while the EVM side always uses the treasury address as the recipient.

Last but not least, the claim logic on OPNET differs on the mode when checking the flow status. In particular, the mint-mode claim ensures the flow status is always active while the release-mode claim allows the flow status to be active and draining. In comparison, the EVM side claim allows active and draining for both mint and release modes.

Recommendation Resolve the above-mentioned inconsistency on both sides so that we can achieve the intended symmetry in the bridge design.

Status The issue has been resolved by this commit: [eb82ade](#).

3.6 Suggested EVM Chain ID/evmBridge Validation in addFlow()

- ID: PVE-006
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: BridgeEscrow, BridgeDepository
- Category: Coding Practices [7]
- CWE subcategory: CWE-563 [4]

Description

As mentioned earlier, a flow is the smallest atom of bridge routing in MaxiBridge. The flow is defined by six fields, i.e., mode, evmChainId, evmBridge, evmToken, opnetBridge, and opnetToken. While reviewing

the logic to add a new flow, we notice current validation logic may be improved.

In the following, we use the EVM-side logic as the example to add a new flow. We notice the `evmChainId` field is always the same as the blockchain's `chainid` and the `evmBridge` field should be the contract itself. In other words, current checks (lines 1926-1927) can be much strengthened. Similarly, the OPNET-side logic can further ensure the `opnetBridge` field is the `BridgeDepository` contract itself.

```

1924     function addFlow(FlowAddParams calldata p) external onlyOwner returns (bytes32
        flowId) {
1925         if (p.mode > uint8(TokenMode.POOLED_LOCK_VEST)) revert FlowInvalidMode();
1926         if (p.evmChainId == 0) revert FlowZeroChainId();
1927         if (p.evmBridge == address(0) p.evmToken == address(0)) revert ZeroAddress();
1928         if (p.opnetBridge == bytes32(0) p.opnetToken == bytes32(0)) revert ZeroAddress
            ();
1929         // E-3 - per-leg decimals are independent (any pair is allowed). The
1930         // claim legs ('claim' modes 0/3/4, 'claimMintWrapped' modes 1/2) scale
1931         // the signed source amount to EVM-destination units via
1932         // AmountPolicy.scale before any accounting (see _grossDst). Bounds
1933         // only: 1..MAX_DECIMALS so '10**delta' stays inside uint256.
1934         if (p.evmDecimals == 0 p.evmDecimals > 30) revert FlowInvalidDecimals();
1935         if (p.opnetDecimals == 0 p.opnetDecimals > 30) revert FlowInvalidDecimals();
1936         if (p.feeBps > MAX_FEE_BPS) revert FeeBpsTooHigh();
1937         if (p.tipCapBps > MAX_TIP_BPS) revert TipCapTooHigh();
1938         // #62-fix - if 'minAmount' is set, it MUST exceed 'minFee'. Otherwise
1939         // the public-facing minimum advertises a usable amount that 'lock'
1940         // would reject with 'FeeExceedsAmount' (because 'fee' >= 'received').
1941         // 'minAmount == 0' is the explicit "no-floor" config and is allowed.
1942         if (p.minAmount > 0 && p.minAmount <= p.minFee) revert FlowMinAmountBelowMinFee
            ();
1943         // Mode 4 (POOLED_LOCK_VEST) is two-step: addFlow registers the route,
1944         // 'setFlowVestingVault' then wires the destination VestingVault before
1945         // any user-facing path becomes safe. The lock and claim paths both
1946         // defensively check 'vestingVault != 0' for mode 4, so a half-set-up
1947         // flow rejects locks/claims rather than silently stranding tokens.
1948
1949         flowId = computeFlowId(
1950             p.mode, p.evmChainId, p.evmBridge, p.evmToken, p.opnetBridge, p.opnetToken
1951         );
1952
1953         // Set-once: an existing record with non-zero chainId means the
1954         // flowId is already taken. Use chainId as the existence flag
1955         // because addFlow rejects chainId==0.
1956         if (flows[flowId].evmChainId != 0) revert FlowAlreadyExists();
1957         ...
1958     }

```

Listing 3.6: BridgeEscrow::addFlow()

```

169     protected override _mint(to: Address, amount: u256): void {
170         if (to === Address.zero()) {
171             throw new Revert('ySLOHM: mint to zero address');
172         }

```

```

174     const gonsPerFrag: u256 = this._gonsPerFragment.value;
175     const gonValue: u256 = SafeMath.mul(amount, gonsPerFrag);

177     // Update total supply
178     const currentSupply: u256 = this._totalSupply.value;
179     this._totalSupply.value = SafeMath.add(currentSupply, amount);

181     // Update total gons tracked
182     const currentTotalGons: u256 = this._totalGons.value;
183     this._totalGons.value = SafeMath.add(currentTotalGons, gonValue);

185     // Allocate gons to recipient
186     const toGons: u256 = this._gonBalances.get(to);
187     this._gonBalances.set(to, SafeMath.add(toGons, gonValue));

189     this.createTransferredEvent(Blockchain.tx.sender, Address.zero(), to, amount);
190 }

```

Listing 3.7: ySLOHM::_mint()

Recommendation Strengthen the logic in the above-mentioned routines to better validate the flow fields.

Status The issue has been resolved by the following commit: 834278f.

3.7 Revisited drainInventoryOpNet() Logic in BridgeDepository

- ID: PVE-007
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: BridgeDepository
- Category: Business Logic [8]
- CWE subcategory: CWE-841 [5]

Description

In Section 3.4, we have examined the inventory provision logic that requires vouchers to validate against the flow's status before possible inventory mutation. In this section, we further examine the logic to drain the inventory from a flow and report a missing check on the flow mode.

Specifically, we show below the implementation of the `drainInventoryOpNet()` function, a governor-gated one to withdraw certain amount of a flow's canonical OP20 inventory from the bridge. It comes to our attention that the inventory may be drained only in the modes of 1/3/4, i.e., `INVERSE_WRAPPED`, `POOLED_LOCK_RELEASE`, and `POOLED_LOCK_VEST`. And the `drainInventory()` counterpart on the EVM sides

can be similarly improved by enforcing the modes of 0/3/4 only, i.e., WRAPPED, POOLED_LOCK_RELEASE, and POOLED_LOCK_VEST.

```

2203     public drainInventoryOpNet(calldata: Calldata): BytesWriter {
2204         this.onlyGovernor();
2205         if (!this._paused.value) {
2206             throw new Revert('BridgeDepository: must pause before drain');
2207         }
2208         const flowId: u256 = calldata.readU256();
2209         const token: Address = calldata.readAddress();
2210         const amount: u256 = calldata.readU256();
2211         const recipient: Address = calldata.readAddress();
2212         if (token.isZero() || recipient.isZero()) throw new Revert('BridgeDepository: zero
                addr');
2213         if (amount.isZero()) throw new Revert('BridgeDepository: zero amount');
2214         if (this._flowExists.get(flowId).isZero()) {
2215             throw new Revert('BridgeDepository: flow not found');
2216         }
2217         if (!u256.eq(this._flowOpnetToken.get(flowId), _opnetAddrToU256(token))) {
2218             throw new Revert('BridgeDepository: token not flow canonical');
2219         }

2221         // Verify -> effect -> interaction (CEI). Ledger debit precedes the
2222         // external transfer; an over-drain reverts before any token moves.
2223         const before: u256 = this._flowInventory.get(flowId);
2224         if (u256.lt(before, amount)) {
2225             throw new Revert('BridgeDepository: insufficient flow inventory');
2226         }
2227         this._flowInventory.set(flowId, SafeMath.sub(before, amount));

2229         const transferSelector: u32 = encodeSelector('transfer(address,uint256)');
2230         const w = new BytesWriter(4 + ADDRESS_BYTE_LENGTH + 32);
2231         w.writeSelector(transferSelector);
2232         w.writeAddress(recipient);
2233         w.writeU256(amount);
2234         Blockchain.call(token, w);

2236         this.emitEvent(
2237             new InventoryDrainedOpNet(flowId, token, recipient, Blockchain.tx.sender,
                amount),
2238         );
2239         return new BytesWriter(0);
2240     }

```

Listing 3.8: BridgeDepository::drainInventoryOpNet()

Recommendation Revise the above-mentioned routines to properly enforce the missing mode checks.

Status The issue has been resolved by the following commits: 3fa6396 and 4b75a7f.

3.8 Arithmetic Overflow Avoidance in VestingVault

- ID: PVE-008
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: VestingVault
- Category: Coding Practices [7]
- CWE subcategory: CWE-1126 [2]

Description

The flow mode of POOLED_LOCK_VEST allows to specify a destination-side vault to hold bridged funds and release them to the beneficiary linearly over a fixed block window. The beneficiary claims accrued portions at their own cadence. In the process of examining the related vesting vault, we notice a possible risk of leading to arithmetic overflow. This risk can be resolved by ensuring the given vesting blocks do not exceed an upper limit.

To demonstrate, we show below the implementation of the related `depositFor()` function in VestingVault. This function is designed to open a new linear-vest schedule on behalf of beneficiary. We notice the ending block of the vest schedule is calculated as `endBlock = startBlock + VESTING_BLOCKS` (line 184), which could be overflowed if the `VESTING_BLOCKS` is sufficiently large without being bounded under 2^{63} .

```

154     function depositFor(address beneficiary, uint256 amount, bytes32 scheduleKey)
155     external
156     nonReentrant
157     onlyBridge
158     {
159         if (beneficiary == address(0)) revert ZeroAddress();
160         if (amount == 0) revert ZeroAmount();
161         if (amount > type(uint128).max) revert AmountOverflow();
162
163         Schedule storage s = schedules[beneficiary][scheduleKey];
164         // total != 0 means an active schedule already exists for this key.
165         // We do NOT permit top-ups - each bridge claim is its own schedule.
166         if (s.total != 0) revert ScheduleAlreadyExists();
167
168         // Pull the tokens. Balance-delta verifies the bridge actually paid
169         // in full (no fee-on-transfer leakage). Reverts loudly on shortfall
170         // rather than opening an under-funded schedule.
171         uint256 balBefore = TOKEN.balanceOf(address(this));
172         TOKEN.safeTransferFrom(msg.sender, address(this), amount);
173         uint256 received;
174         unchecked {
175             received = TOKEN.balanceOf(address(this)) - balBefore;
176         }
177         if (received != amount) revert TokenTransferIncomplete();
178     }

```

```

179     uint64 startBlock = uint64(block.number);
180     uint64 endBlock;
181     unchecked {
182         // VESTING_BLOCKS <= 2^64-1 by type; block.number realistically <<
183         // 2^63, so the sum cannot overflow uint64.
184         endBlock = startBlock + VESTING_BLOCKS;
185     }
186
187     // forge-lint: disable-next-line(unsafe-typecast) - AmountOverflow() guard at
        function start
188     s.total = uint128(amount);
189     s.startBlock = startBlock;
190     s.endBlock = endBlock;
191
192     // forge-lint: disable-next-line(unsafe-typecast) - AmountOverflow() guard at
        function start
193     emit Deposited(beneficiary, scheduleKey, uint128(amount), startBlock, endBlock);
194 }

```

Listing 3.9: VestingVault::depositFor()

Similarly, the provisionInventory() function in BridgeEscrow may be improved by enforcing the newInventory variable to be smaller than type(uint128).max.

Recommendation Improve the above routine to block the possible risk of resulting in an arithmetic overflow.

Status The issue has been resolved by the following commit: 3fa6396.

3.9 Improved mintTo() Logic in WrappedOP20

- ID: PVE-009
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: WrappedOP20
- Category: Business Logic [8]
- CWE subcategory: CWE-841 [5]

Description

MaxiBridge provides a canonical wrapped representation in the form of WrappedOP20 for an EVM asset on OPNet. Within the WrappedOP20 contract, there is a pause flag defined as _paused. The governor can use this pause flag to halt burnForRelease so incident response, if any, can stop new EVM release liabilities from accruing while the BridgeDepository is also paused. Our analysis shows that this pause flag is currently honored in burnForRelease(), but not in mintTo(). We suggest to honor the pause flag inside mintTo() as well since the authorized minter may not be limited to the BridgeDepository

only. Note this is different from the EVM counterpart where the only authorized entity to call `mintTo()` is `BRIDGE`.

```

416     public mintTo(calldata: Calldata): BytesWriter {
417         this.onlyMinter();
418         const to: Address = calldata.readAddress();
419         const amount: u256 = calldata.readU256();
420         if (to.isZero()) {
421             throw new Revert('WrappedOP20: mint to zero');
422         }
423         if (amount.isZero()) {
424             throw new Revert('WrappedOP20: zero amount');
425         }
426         this._mint(to, amount);
427         return new BytesWriter(0);
428     }

```

Listing 3.10: `WrappedOP20::mintTo()`

Recommendation Improve the above routine to ensure the pause flag is also honored when new OP20 tokens are minted.

Status The issue has been resolved by the following commit: `cc996e7`.

3.10 Trust Issue of Admin Keys

- ID: PVE-010
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: Multiple Contracts
- Category: Security Features [6]
- CWE subcategory: CWE-287 [3]

Description

In the `MaxiBridge` protocol, there is a privileged admin account that plays a critical role in governing and regulating the protocol-wide operations (e.g., configure/gate parameters, manage signers, and pause/unpause/upgrade the protocol contract implementations). Our analysis shows that the privileged account needs to be scrutinized. In the following, we use the `BridgeEscrow` contract as an example and show the representative functions potentially affected by the privileged account.

```

1466     function addSigner(address newSigner) external onlyOwner {...}
1468     function removeSigner(address oldSigner) external onlyOwnerOrGuardian {...}
1470     function setThreshold(uint256 newThreshold) external onlyOwner {...}

```

```

1472    /// @notice Atomic add+remove+threshold update - required for safe
1473    ///         transitions like 1-of-1 -> 2-of-2 (no intermediate weakening).
1474    function migrateSignerSet(
1475        address[] calldata addList,
1476        address[] calldata removeList,
1477        uint256 newThreshold
1478    ) external onlyOwnerOrGuardian {...}

1480    function cancelVoucher(bytes32 opnetNonce) external onlyOwnerOrGuardian {...}
1481    ...
1482    // -----
1483    // Admin - token allowlist + pause + treasury/guardian
1484    // -----

1486    function setSupportedToken(address token, bool enabled) external onlyOwner {...}

1488    /// @notice Protective freeze. Callable by owner, guardian, OR the
1489    ///         dedicated 'pauser' role - the widest incident-response surface.
1490    ///         'unpause' deliberately stays narrower (owner/guardian only).
1491    function pause() external {...}

1493    /// @notice Resume. OWNER-ONLY (audit H-01). The guardian and 'pauser'
1494    ///         roles can freeze for incident response but must never re-open
1495    ///         the bridge - resumption is a deliberate governance decision
1496    ///         (timelock-gated on mainnet), so a compromised incident-response
1497    ///         key cannot thaw a legitimate freeze.
1498    function unpause() external onlyOwner {...}

1500    /// @notice Set/rotate/disable the dedicated pause role.
1501    /// @dev    Owner-only (timelock-gated on mainnet). Zero address disables.
1502    function setPauser(address newPauser) external onlyOwner {...}

1504    /// @notice Set or rotate the emergency-drain destination.
1505    /// @dev    Owner-rotatable (was set-once pre-roles PR). On mainnet 'owner'
1506    ///         is the 3-day TimelockController, so rotation is timelock-gated
1507    ///         (the timelock IS the staging step). H-3 (audit 2026-05-27):
1508    ///         emits the OLD treasury alongside the new for full audit-trail
1509    ///         readability - a forgotten or accidental rotation now shows up
1510    ///         with both values in the indexer. If EOA owners are ever used,
1511    ///         consider adding an explicit 'startTreasuryTransfer' /
1512    ///         'acceptTreasury' 2-step on top (see 'treasury' storage doc).
1513    function setTreasury(address newTreasury) external onlyOwner {...}

1515    /// @notice Set or rotate the guardian (incident-response co-signer).
1516    /// @dev    Owner-rotatable (was set-once pre-roles PR). Timelock-gated on
1517    ///         mainnet (3 days). H-3 - emits old + new for audit-trail parity
1518    ///         with 'setTreasury'. Same 2-step follow-up consideration applies.
1519    function setGuardian(address newGuardian) external onlyOwner {...}

```

Listing 3.11: Example Privileged Operations in BridgeEscrow

We emphasize that the privilege assignment with various protocol contracts is necessary and

required for proper protocol operations. However, it is worrisome if the admin account is not governed by a DAO-like structure. The discussion with the team has confirmed that the governance will be managed by a multi-sig account. And a number of security-sensitive operations are behind a timelock.

Moreover, it should be noted that current contracts have the support of being upgraded after the deployment. And there is a need to properly manage the deployment key for upgrade as they fall in this trust issue as well.

Recommendation Promptly transfer the privileged account to the intended governance contract. All changes from privileged operations may need to be mediated with necessary timelocks.

Status This issue has been resolved as changes from privileged operations have been mediated with use of multisig and necessary timelocks.



4 | Conclusion

In this audit, we have analyzed the design and implementation of the MaxiBridge protocol, which is a general-purpose token bridge between Ethereum mainnet and OPNet (Bitcoin L1). Governance can onboard almost any token pair through five flow modes (wrapped mint/burn, pooled lock/release, pooled lock/vest) with decimal-aware accounting. Depending on the flow's mode, each side does lock-and-release, mint-and-burn, or pooled custody. Both directions use a voucher model: the user pays gas on the destination chain, not the server. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Meanwhile, we need to emphasize that AssemblyScript-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.



References

- [1] MITRE. CWE-1099: Inconsistent Naming Conventions for Identifiers. <https://cwe.mitre.org/data/definitions/1099.html>.
- [2] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. <https://cwe.mitre.org/data/definitions/1126.html>.
- [3] MITRE. CWE-287: Improper Authentication. <https://cwe.mitre.org/data/definitions/287.html>.
- [4] MITRE. CWE-563: Assignment to Variable without Use. <https://cwe.mitre.org/data/definitions/563.html>.
- [5] MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. <https://cwe.mitre.org/data/definitions/841.html>.
- [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
- [7] MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- [8] MITRE. CWE CATEGORY: Business Logic Errors. <https://cwe.mitre.org/data/definitions/840.html>.
- [9] MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[11] PeckShield. PeckShield Inc. <https://www.peckshield.com>.

